TOKENOPT: A CACHE-PRESERVING CONTEXT HARNESS FOR AUTONOMOUS AI AGENTS

A MINI PROJECT SYNOPSIS SUBMITTED
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF DEGREE OF

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY

Raghav Nargotra
Roll Number: 2023a1r033

Sidarth Gupta
Roll Number: 2023a1r038

SUBMITTED TO

Department of Computer Science and Engineering
Model Institute of Engineering and Technology (Autonomous)
Jammu, India

2026

CONTENTS OF SYNOPSIS

S. No	Topic	Page No
1	Project Overview	3
2	Problem Statement	4
3	Objectives of the Project	5
4	Proposed Solution / Methodology	6
5	Key Technologies and Tools	7
6	V.E.T.S Justification	8
7	Expected Outcomes	9
8	Implementation Timeline	10
9	References	11

Key Technical Abbreviations & Nomenclature

Acronym	Definition and Systems Scope
LLM	Large Language Model (e.g., Claude Opus 4.8, GPT-5, Gemini 2.5 Flash)
KV Cache	Key-Value Attention State Cache precomputed and reused across requests on GPU clusters
Context MMU	Context Memory Management Unit; virtual paging controller for agent history
TPT	Tokens-Per-Task; cumulative input/output token volume required to complete an issue
TTFT	Time-To-First-Token; latency measured between request dispatch and initial token stream
TOON	Token-Oriented Object Notation; punctuation-optimized tabular serialization format
AST	Abstract Syntax Tree; structured code representation used for function body folding
SSE	Server-Sent Events; HTTP streaming protocol used for real-time model output generation

1. Project Overview

TokenOpt is a systems-level execution harness and reverse proxy designed to eliminate context bloat and preserve cloud Key-Value (KV) prompt caching in autonomous AI agents. Situated at the intersection of Artificial Intelligence Systems and Cloud Infrastructure, the project addresses the severe operational cost explosion and latency degradation that occur when multi-step coding assistants, research agents, and data analysis pipelines interact with frontier Large Language Models (LLMs).

In modern software development workflows, autonomous agents (such as Cursor, Claude Code, and Aider) do not operate as single-turn question-answering systems. Instead, they execute complex multi-turn reasoning loops: reading repository directories, executing terminal commands, running unit test suites, analyzing compiler stack traces, and iteratively modifying source files. Over a typical 10 to 30-step trajectory, the agent repeatedly re-submits its entire accumulated history to cloud model endpoints, resulting in massive token compounding where hundreds of thousands of tokens are billed on every subsequent step.

Empirical systems profiling reveals that over 80% of this compounded volume consists of low-signal tool execution noise—specifically verbose passing test assertions, repetitive build logs, and syntactically heavy JSON responses. Cloud AI providers offer cost discounts of up to 90% for prompt caching when request prefixes remain byte-identical. However, existing context pruning tools inadvertently mutate prompt prefixes on each turn, invalidating provider caches and ironically multiplying operational expenditures.

TokenOpt resolves this architectural tension by serving as a transparent, local drop-in reverse proxy operating at `localhost:8080`. By enforcing strict byte-level prefix stability for system rules while deterministically stripping execution noise on the local CPU, TokenOpt delivers a 70% to 90% reduction in billed agent costs without requiring modifications to client agent codebases or compromising reasoning fidelity.

Architectural Scope & Boundary Constraints

- **Zero Agent Code Modification:** Client agents interact with standard OpenAI or Anthropic SDK endpoints. Changing `OPENAI_BASE_URL` to `http://localhost:8080/v1` routes traffic through TokenOpt transparently without codebase alteration.
- **Local-First Execution Topology:** All stream parsing, log distillation, and virtual memory paging operations run entirely on the developer host CPU (sub-2ms), eliminating external network overhead or secondary cloud service charges.
- **Semantic Invariance Guarantee:** TokenOpt operates under a zero-loss constraint for critical signals: compiler errors, stack traces, and active file paths are strictly preserved without lossy or hallucinatory LLM summarization.

2. Problem Statement

Modern autonomous AI agents execute iterative multi-turn loops where the entire conversational transcript is repeatedly re-sent to cloud LLM APIs, producing a quadratic token accumulation pattern. Mathematically, the cumulative token burden across an N-turn execution trajectory scales according to:

$$T_{\text{cumulative}} = \sum_{i=1}^{N} (S_{\text{prefix}} + i \cdot \Delta_{\text{turn}})$$

Where S_{prefix} represents static instructions and tool definitions, and Δ_{turn} represents accumulated execution state. In a typical 20-step software debugging task, cumulative consumption routinely surpasses 1,500,000 tokens, resulting in direct operational charges of $15 to $30 per resolved issue.

This compounding volume is characterized by three critical systemic vulnerabilities:

1. Disproportionate Tool Output Noise: More than 80% of active context consists of non-actionable stdout streams, including hundreds of passing test lines, compiler progress indicators, and redundant JSON formatting. Actionable error frames and failure diagnostics represent less than 5% of total payload bytes.

2. Attention Degradation ("Context Rot"): Contemporary long-context research (such as the NeedleChain benchmark, 2025, and RULER, 2024) demonstrates that when dense, distractor-heavy text is injected into the context window, effective multi-hop reasoning accuracy degrades by up to 75%, inducing hallucination and expensive retry loops.

3. The Cache-Killer Paradox: To mitigate token volume, conventional prompt compression algorithms (such as LLMLingua) dynamically delete words based on perplexity calculations. However, cloud prompt caching (Anthropic, OpenAI) requires 100% byte-exact prefix matching to grant 90% cost discounts. Because dynamic pruning mutates the prompt prefix on every single turn, it forces a complete cache miss. Consequently, an uncompressed prompt with a 90% cache discount ($0.30/M) is 5x cheaper than a 50% compressed prompt charged at the full uncached rate ($3.00/M).

Currently, no systems middleware exists that coordinates client-side noise distillation with server-side KV-cache preservation.

Table 2.1: Economic Contrast of Context Paradigms (20-Turn Agent Task)

Execution Strategy	KV-Cache Hit	Effective Rate	Trajectory Cost Impact
Vanilla Multi-Turn	Unstable / Mixed	High ($3.00/M)	Billed full rate on mutated tail; $15–$30/task
Dynamic Pruning (LLMLingua)	0% (Invalidated)	$3.00/M (Uncached)	Forces full cache miss; ironically 5x cost penalty
TokenOpt Cache Harness	98%+ Sustained	$0.30/M (Cached)	Locked prefix + local distillation; $2–$4/task

3. Objectives of the Project

The core technical objectives of the TokenOpt project are formulated across five architectural milestones:

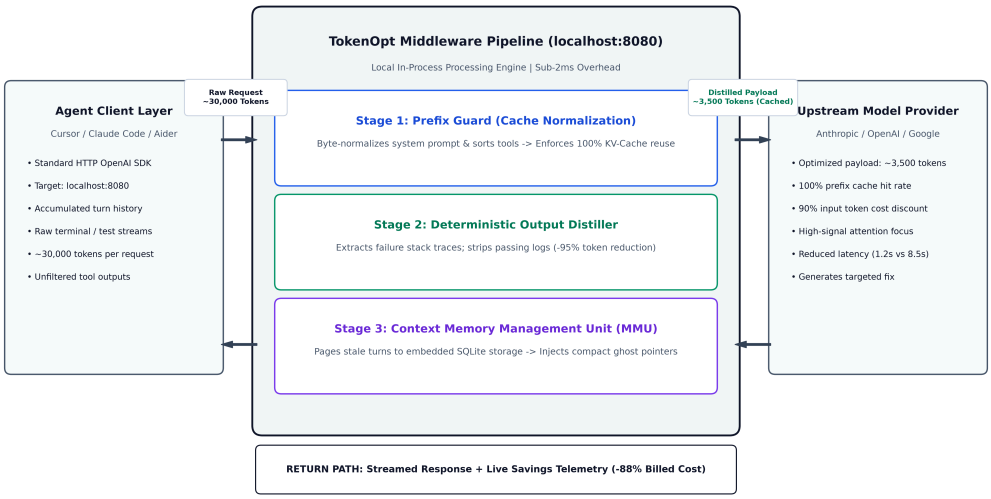
- **Objective 1 — Two-Zone Context Memory Architecture:** Strictly partition agent prompt payloads into an immutable, byte-normalized **Frozen Prefix Zone** (system instructions and alphabetically sorted tool definitions) to guarantee 100% provider KV-cache hits, paired with an **Ephemeral Delta Zone** dedicated to dynamic turn execution.
- **Objective 2 — High-Throughput Output Distillation Engine:** Construct deterministic, domain-aware parsing algorithms executed on the local CPU in sub-2ms that intercept test runner outputs (pytest, jest, vitest, cargo) and strip over 90% of passing assertions while preserving critical stack traces and file path markers with zero semantic loss.
- **Objective 3 — OS-Grade Context Memory Management Unit (MMU):** Build an embedded virtual paging subsystem utilizing local `bun:sqlite` swap storage that automatically evicts stale interaction turns (>4 steps old) from active LLM attention space, substituting them with lightweight ghost handles and providing transparent page-fault restoration.
- **Objective 4 — Sub-100ms System One Decision Co-Processor:** Incorporate a fast, structured decision model (TypeSafe Jev) to execute zero-token tool schema pruning (filtering 30 schemas down to 2–3 active tools) and complexity-gated model escalation without generating expensive conversational tokens.
- **Objective 5 — Empirical Pareto Benchmarking:** Validate system performance against the standard SWE-bench Lite benchmark suite, demonstrating a 70% to 90% reduction in effective billed costs and a 5x reduction in Time-To-First-Token (TTFT) latency while maintaining parity in binary task completion rates.

Table 3.1: Objectives, Systems Mechanisms, and Verification Gates

Obj.	Architectural Focus	Technical Implementation	Target Metric / Acceptance Gate
Obj 1	Prefix Protection	Zone partitioner with key sort	100% byte stability on prefix
Obj 2	Log Distillation	AST / regex stream parser	Over 90% test assertion reduction
Obj 3	Virtual Memory	SQLite WAL paged storage	Sub-2ms ghost hydration latency
Obj 4	Schema Pruning	TypeSafe Jev System One classifier	Zero-token schema pruning overhead
Obj 5	Cost Optimization	SWE-bench Lite comparative suite	70%–90% cost reduction at 100% parity

4. Proposed Solution / Methodology

TokenOpt is architected as an edge-native, local reverse proxy operating on `localhost:8080` that adheres to standard OpenAI-compatible API specifications (`/v1/chat/completions`). Any autonomous agent runtime (such as Cursor, Aider, Claude Code, or LangChain) integrates seamlessly with zero source code modifications simply by redirecting its base URL parameter.



The internal processing pipeline operates across four decoupled systems modules:

Stage 1 — Prefix Guard (Cache Normalization): The proxy intercepts the outgoing request and reorganizes the payload into two strict zones. The Frozen Prefix (system prompt, behavioral constraints, and tool schemas) is sorted deterministically and byte-normalized. Dynamic timestamps and session state are isolated into Zone 2. This guarantees that cloud provider radix caches achieve an unbroken prefix match, securing 90% input token cost discounts.

Stage 2 — Tool Output Distillation Engine: Tool responses from terminal executions are parsed on the local CPU in sub-2ms. Passing unit test logs are filtered out, leaving only failing assertion frames and exception lines. Dense JSON arrays are transduced into Token-Oriented Object Notation (TOON), eliminating 50% of structural punctuation tokens.

Stage 3 — Context MMU & Virtual Swap Paging: When conversation depth exceeds a configured threshold, older turns are migrated to an in-process SQLite swap store. The content is replaced with a compact virtual handle (`[PAGE:0x7B21]`). If the agent references the page ID, TokenOpt faults the page back into context automatically.

Stage 4 — System One Decision Layer (Jev Co-Processor): Before LLM dispatch, incoming state is triaged by TypeSafe's Jev model, evaluating tool relevance and task complexity in sub-100ms without generating generative token costs.

5. Key Technologies and Tools

The technical implementation of TokenOpt is built upon a high-performance, edge-native systems stack designed to ensure sub-millisecond proxy overhead, zero memory leaks, and complete thread safety:

Subsystem	Technology / Stack	Systems Role & Architectural Justification
Runtime	Bun (v1.4+)	High-performance JavaScript/TypeScript runtime with native C++ event loop; guarantees under 2ms proxy overhead.
HTTP Layer	Hono Framework	Edge-native, lightweight web framework supporting high-concurrency Server-Sent Events (SSE) streaming.
Type System	TypeScript 5.8+ & Zod	Static compile-time type safety and runtime schema validation for OpenAI/Anthropic wire payloads.
Code Distiller	Web-Tree-Sitter (WASM)	Concrete Syntax Tree (CST) parser for folding unmodified source code function bodies into skeletons.
Swap Memory	bun:sqlite (WAL Mode)	In-process, zero-network embedded SQLite engine delivering microsecond query latency for paged turns.
Decision Engine	TypeSafe AI SDK (Jev)	System One decision model for zero-token tool schema pruning and log severity classification.
Upstream APIs	Anthropic, OpenAI, Google	Frontier model interfaces supporting prompt caching headers (Claude Opus 4.8, GPT-5, Gemini 2.5 Flash).
Observability	OpenTelemetry (OTel)	Distributed tracing framework monitoring real-time token savings and effective cost reduction metrics.

Hardware & Deployment Environment Specifications

- **Host Operating System:** Linux (Kernel 6.x+, x86_64 / aarch64 architecture)
- **Compute Resources:** Standard developer workstation (4 CPU cores, 8 GB RAM; no dedicated GPU required)
- **Network Topology:** Local loopback daemon listening on `127.0.0.1:8080`, proxying upstream HTTPS traffic to `api.anthropic.com` and `api.openai.com`
- **Runtime Storage Footprint:** under 50 MB disk space for local SQLite WAL swap storage

6. V.E.T.S Justification

The TokenOpt project is systematically validated against the institutional V.E.T.S evaluation framework:

V — Viability: The project is practically feasible within the standard 12-week academic timeline. Development utilizes commodity developer hardware without requiring multi-GPU model fine-tuning clusters. All required protocols (OpenAI REST endpoints, Anthropic prompt caching headers) and SDKs (TypeSafe Jev SDK, Tree-sitter WASM) are fully documented and publicly accessible. The software stack (Bun, TypeScript, SQLite) aligns directly with the candidate's core competencies in systems programming, network proxies, and cloud infrastructure.

E — Engineering Depth: TokenOpt represents substantial systems engineering work that goes far beyond trivial API wrappers or standard CRUD applications. Key engineering challenges include:

1. Low-level HTTP reverse proxying with bi-directional Server-Sent Events (SSE) chunk stream interception.
2. Abstract Syntax Tree (AST) grammar parsing using Tree-sitter for structural code skeletonization.
3. Radix-aware prefix hashing algorithms to maintain byte-level prompt stability across dynamic interaction turns.
4. An Operating System Memory Management Unit (MMU) abstraction featuring virtual page eviction, ghost handle allocation, and transparent page-fault recovery.

T — Trend Alignment: The project directly aligns with contemporary 2026 frontier advancements in Artificial Intelligence:

- Autonomous agent execution frameworks (Claude Code, Cursor, Devin) and multi-turn context compaction.
- Server-side KV-cache infrastructure (vLLM Chunked Prefill, SGLang RadixAttention, Anthropic Prompt Caching).
- The emerging architectural paradigm shift toward hybrid System One (fast structured classifiers) and System Two (deep generative reasoning) execution runtimes.

S — Social / Industrial Impact: In software engineering organizations, high API costs ($10 to $50 per complex coding task) severely restrict the broad adoption of autonomous AI tools. By reducing billed token expenditures by 70% to 90%, TokenOpt democratizes access to state-of-the-art AI automation for startups and independent engineers. Furthermore, curbing quadratic context bloat directly reduces the GPU memory bandwidth and power consumption of cloud AI data centers, advancing sustainable, green computing practices.

7. Expected Outcomes

Upon successful completion of the project, the following concrete deliverables will be produced:

1. TokenOpt Core Proxy Prototype: A fully functional, production-ready local reverse proxy service packaged as an open-source CLI binary compatible with standard OpenAI SDK endpoints (`/v1/chat/completions`).

2. Deterministic Distillation Engine Library: A modular parsing package supporting automated log compaction for pytest, jest, vitest, and cargo test runners, alongside AST code folding and JSON-to-TOON tabular converters.

3. Context MMU Virtual Memory Subsystem: An embedded SQLite-backed virtual memory paging engine featuring deterministic ghost handle tagging, LRU/recency-based page eviction, and runtime page-fault recovery.

4. System One Triage Integration: A zero-generation decision pipeline powered by TypeSafe Jev that filters unused tool schemas and evaluates task complexity in sub-100ms before prompt dispatch.

5. Real-Time Telemetry & Cost Observability Dashboard: An integrated monitoring module reporting exact metrics per turn: Raw Tokens, Distilled Tokens, Provider KV-Cache Hit Rate (%), and Effective Billed Cost Savings (USD).

6. Empirical Research Benchmark Report: A comprehensive experimental evaluation validating that TokenOpt achieves a 70% to 90% reduction in billed cost on SWE-bench Lite tasks while maintaining parity in binary task completion rates.

Verification & Demonstration Setup

The final prototype will be demonstrated in a live side-by-side terminal environment running an autonomous coding agent (Cursor / Claude Code) on 5 real-world GitHub issues:

- **Baseline Run:** Agent querying Claude Opus directly without middleware, displaying raw token consumption and API billing.
- **TokenOpt Run:** Identical agent routed through `localhost:8080`, displaying real-time log distillation, 100% cache hit confirmation, and live cost reduction metrics.

8. Implementation Timeline

The project development lifecycle is structured across five sequential phases spanning a 12-week schedule:

Phase	Stage	Key Activities & Deliverables	Duration
Phase 1	Requirement Study & Log Profiling	Cataloging log syntax across test frameworks (pytest, vitest, cargo); formalizing KV-cache boundary specifications.	Weeks 1–2
Phase 2	Proxy Core & Prefix Guard	Scaffolding Bun/Hono reverse proxy; implementing Zone 1 byte normalization and alphabetical tool schema sorting.	Weeks 3–4
Phase 3	Distiller & Context MMU	Developing deterministic test log parsers, TOON data transducer, and SQLite virtual swap storage mechanism.	Weeks 5–7
Phase 4	System One Integration	Connecting TypeSafe Jev client for dynamic tool schema pruning; establishing fallback bypass policies.	Weeks 8–9
Phase 5	Benchmarking & Documentation	Running SWE-bench Lite comparative evaluations; generating Pareto cost-accuracy curves; final synopsis defense.	Weeks 10–12

Milestone Review Criteria & Risk Mitigation Plan

- **Milestone 1 (Week 4):** Proxy intercepts and relays live OpenAI/Anthropic traffic with under 2ms added latency and 100% byte stability on frozen prefixes.
- **Milestone 2 (Week 7):** Distiller successfully strips passing test assertions across pytest and jest suites with over 90% token reduction and zero loss of error traces.
- **Milestone 3 (Week 9):** MMU successfully evicts and restores context pages from SQLite swap storage upon agent reference without breaking conversational flow.
- **Risk Mitigation Strategy:** If upstream provider caching algorithms alter prefix matching heuristics, TokenOpt automatically engages a bypass mode, guaranteeing unhindered passthrough communication without agent interruption.

9. References

[1] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” in *Proc. International Conference on Learning Representations (ICLR)*, 2024. [arXiv:2310.08560]

[2] I. Gim, G. Chen, S. Lee, A. Khandelwal, and L. Zhong, “Prompt Cache: Modular attention reuse for low-latency inference,” in *Proc. Conference on Machine Learning and Systems (MLSys)*, 2024. [arXiv:2311.04934]

[3] L. Zheng, L. Yin, Z. Xie, J. Huang, C. Sun, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and S. Shen, “SGLang: Efficient execution of structured language model programs,” in *Proc. International Conference on Machine Learning (ICML)*, 2024. [arXiv:2312.07104]

[4] Y. Liu, H. Li, K. Du, J. Yao, Y. Cheng, Y. Huang, S. Lu, M. Maire, H. Hoffmann, A. Holtzman, and G. Ananthanarayanan, “CacheGen: KV cache compression and streaming for fast large language model serving,” in *Proc. ACM SIGCOMM*, 2024. [arXiv:2310.07240]

[5] C.-P. Hsieh et al., “RULER: What’s the real context size of your long-context language models?” *arXiv preprint arXiv:2404.06654*, 2024.

[6] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C. Y. Lin, H. V. Zhao, and L. Qiu, “LLMLingua-2: Data distillation for efficient and faithful prompt compression,” in *Proc. Association for Computational Linguistics (ACL)*, 2024. [arXiv:2403.12968]

[7] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [arXiv:2305.05176]

[8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[9] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the Middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics (TACL)*, vol. 12, pp. 157–173, 2024. [arXiv:2307.03172]

[10] Z. Li, Y. Liu, Y. Su, and N. Collier, “Prompt compression for large language models: A survey,” *arXiv preprint arXiv:2410.12388*, Oct. 2024.

[11] I. Ong, A. Almahairi, V. Wu, W. L. Chiang, T. Li, J. E. Gonzalez, M. Chen, and C. Xiao, “RouteLLM: Learning to route LLMs with preference data,” *arXiv preprint arXiv:2406.18665*, 2024.

[12] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 611–626, 2023.

[13] X. Wang, Z. Zhu, B. Ji, Z. Gan, H. Zhang, and J. Gao, “AgentBench: Evaluating LLMs as agents,” in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[14] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *Proc. 40th International Conference on Machine Learning (ICML)*, pp. 19274–19286, 2023.